

Теоретический материал

Двоичный (бинарный) поиск (также известен как метод деления пополам или дихотомия) — классический алгоритм поиска элемента в отсортированном массиве, использующий дробление массива на половины.

Алгоритм включает в себя следующие шаги:

1. Определение значения элемента в середине массива. Полученное значение сравнивается с искомым элементом.
2. Если искомый элемент меньше значения середины, то поиск осуществляется в первой половине элементов, иначе — во второй.
3. Поиск сводится к тому, что вновь определяется значение срединного элемента в выбранной половине и сравнивается с искомым элементом.
4. Процесс продолжается до тех пор, пока не будет найден элемент, равный искомому, или не станет пустым интервал для поиска.

Существует ряд особенностей, которые необходимо учитывать при программной реализации кода:

1. Пусть `first` и `last` – индексы элементов, являющихся левым и правым концом отрезка поиска на некотором шаге алгоритма. В случае больших массивов код $(first + last) / 2$ для поиска середины отрезка может быть ошибочен, если `first` и `last` по отдельности умецаются в свой тип, а `first+last` — нет. Для обхода этого ограничения можно использовать выражение $first + (last - first) / 2$, которое приведет к такому же результату, но позволит не выйти за границы типа данных.
2. Необходимо тестировать код для случаев пустого массива. Массива из одного элемента и другие частные случаи в зависимости от условий задачи.
3. Массив может содержать несколько экземпляров искомого элемента. В зависимости от условия задачи, необходимо учитывать требования поиска первого экземпляра элемента, последнего экземпляра элемента, произвольного экземпляра и т.п.

Пример 1.1

Задача:

Написать программу для бинарного поиска элемента в отсортированном массиве

Решение:

```
1 def binary_search(list, key):
2     low = 0
3     high = len(list) - 1
4
5     while low <= high:
6         mid = (low + high) // 2
7         midVal = list[mid]
8         if midVal == key:
9             return mid
10        if midVal > key:
11            high = mid - 1
12        else:
13            low = mid + 1
14    return 'Элемент не найден'
15
16 a = [1, 2, 3, 4, 5, 6, 7]
17 print(binary_search(a, 3))
```

Ответ:

```
2
...Program finished with exit code 0
Press ENTER to exit console.
```

Задание 1.1

Задача:

Написать программу для бинарного поиска элемента в отсортированном массиве. В случае, если элемент не найден, указать позицию, в которую можно выполнить вставку элемента (чтобы массив оставался отсортированным) и выполнить вставку

Решение:

```

def binary_search(arr, key):
    l = 0
    r = len(arr) - 1

    while l ≤ r:
        mid = (l + r) // 2
        midVal = arr[mid]
        if midVal == key:
            return mid
        if midVal > key:
            r = mid - 1
        else:
            l = mid + 1
    arr.insert(l, key)
    return [l, arr]

```

Ответ:

```

[1, 3, 4, 5, 6, 7], 4 => 2
[1, 3, 4, 5, 6, 7], 2 => [1, [1, 2, 3, 4, 5, 6, 7]]

```

Задание 1.2

Задача:

Массив arr называется горным, если выполняются следующие свойства:

- 1) В массиве не менее трех элементов
- 2) Существуют i ($0 < i < \text{arr.length} - 1$) что:
 - $\text{arr}[0] < \text{arr}[1] < \dots < \text{arr}[i - 1] < \text{arr}[i]$
 - $\text{arr}[i] > \text{arr}[i + 1] > \dots > \text{arr}[\text{arr.length} - 1]$

Иными словами, в массиве есть пик (или несколько пиков) такие, что остальные элементы убывают влево и вправо относительно пика.

Используя алгоритм бинарного поиска, проверить, является ли массив arr горным. Если да – вернуть индекс первого пика.

Решение:

```
def is_picky(arr):  
    l = get_pick(arr, 4)  
    m = get_pick(arr, 2)  
    r = get_pick(arr, 1.5)  
    if (l == m and l != None) or (m == r and m != None) or (l == r and l != None):  
        return f"Массив горный: {m}"  
    else:  
        return "Массив не горный"
```

```
def get_pick(arr, coefficient):  
    l = 0  
    r = len(arr) - 1  
    mid = int((r + l) // coefficient)  
  
    while True:  
        if (  
            mid + 1 == len(arr)  
            or mid - 1 == -1  
            or (arr[mid - 1] > arr[mid] and arr[mid] < arr[mid + 1])  
        ):  
            return None  
        elif arr[mid - 1] < arr[mid] and arr[mid] > arr[mid + 1]:  
            return mid  
        elif arr[mid - 1] < arr[mid] < arr[mid + 1]:  
            l = mid + 1  
        elif arr[mid - 1] > arr[mid] > arr[mid + 1]:  
            r = mid - 1  
        else:  
            l += 1  
            mid = (r + l) // 2
```

Ответ:

Ввод	Вывод
[1, 2, 3, 4, 1]	Массив горный: 3
[1, 2, 1]	Массив горный: 1
[1, 2, 3, 4]	Массив не горный
[4, 3, 2, 1]	Массив не горный
[1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1]	Массив горный: 9
[0, 0, 2, 1, 2, 0]	Массив не горный
[0, 0, 0]	Массив не горный

Задание 1.3

Задача:

Массив отсортирован по возрастанию (элементы могут повторяться). С помощью алгоритма бинарного поиска выяснить, каких чисел в массиве больше – положительных или отрицательных. Если больше положительных чисел – вывести их количество, если же больше отрицательных, то вывести их количество

Решение:

```

4  ✓ def comparison(neg, pos):
5  ✓     if neg > pos:
6  ✓         return f"Отрицательных больше: {neg}"
7  ✓     if neg < pos:
8  ✓         return f"Положительных больше: {pos}"
9  ✓     return f"Одинаково: {neg}"
10
11
12  ✓ def find_neg_or_pos_more(arr):
13  ✓     l = 0
14  ✓     r = len(arr) - 1
15
16  ✓     while l ≤ r:
17  ✓         mid = (l + r) // 2
18  ✓         if arr[mid] == 0:
19  ✓             neg = len([a for a in arr[:mid] if a ≠ 0])
20  ✓             pos = len([a for a in arr[mid + 1 :] if a ≠ 0])
21  ✓             return comparison(neg, pos)
22  ✓         if arr[mid] > 0:
23  ✓             r = mid - 1
24  ✓         else:
25  ✓             l = mid + 1
26  ✓             neg = r + 1
27  ✓             pos = len(arr) - neg
28  ✓             return comparison(neg, pos)
29

```

Ответ:

Ввод	Вывод
[0, 1, 2]	Положительных больше: 2
[-3, 7, 8, 10]	Положительных больше: 3
[-10, -8, -3, 0]	Отрицательных больше: 3
[-17, -10, -8]	Отрицательных больше: 3
[-4, 0, 0, 0, 10, 11]	Положительных больше: 2
[-7, -6, -5, -4, 0, 1, 3, 5, 6, 7]	Положительных больше: 5
[-3, 3]	Одинаково: 1
[-3, 0, 3]	Одинаково: 1
[-3, -2, -1, 0, 0, 1, 2, 3]	Одинаково: 3
[0, 0, 0]	Одинаково: 0

Задание 1.4*

Задача:

Дан целочисленный массив `nums`. Постройте целочисленный массив `counts`, где `counts[i]` — количество меньших элементов справа от `nums[i]`.

Пример:

Ввод: `nums = [5,2,6,1]`

Вывод: `[2,1,1,0]`

Объяснение:

Справа от 5 находятся 2 меньших элемента (2 и 1).

Справа от 2 находится только 1 элемент меньшего размера (1).

Справа от 6 находится 1 элемент поменьше (1).

Справа от 1 находится 0 элементов меньшего размера.

<i>Решение:</i>	
	<pre> ✓ def get_counts(arr): counts = [] ✓ for i in range(len(arr)): count = 0 ✓ for j in range(i + 1, len(arr)): ✓ if arr[i] > arr[j]: count += 1 counts.append(count) return counts </pre>
<i>Ответ:</i>	
	<pre> [5, 2, 6, 1] => [2, 1, 1, 0] [0, 1, 2, 3, 4, 5, 6] => [0, 0, 0, 0, 0, 0, 0] [6, 5, 4, 3, 2, 1] => [5, 4, 3, 2, 1, 0] [6, -3, 7, -1, -5, 9, 0, 4, 3] => [6, 1, 5, 1, 0, 3, 0, 1, 0] [1, 2, 3, 4] => [0, 0, 0, 0] [5, 5, 5, 5] => [0, 0, 0, 0] </pre>